

Reviewer Pack — Minimal Local Induction Dimension over Boolean Ring and Comm...

Entry 5b9ce10bce33 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 04:43:53 UTC

Minimal Local Induction Dimension over Boolean Ring and Communication Complexity Rank

Entry ID: 5b9ce10bce33 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 04:43:53 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Local Induction Dimension in Boolean Algebra) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every instance of a communication complexity problem with matrix rank R , the minimal local induction dimension of the associated hyperplane arrangement over the Boolean ring is upper bounded by R^2 .

Rationale (proposer's reasoning):

The use of Local Induction Dimension in Boolean Algebra provides a topological structure that could capture the inherent complexity of communication problems. This conjecture suggests that the complexity of communication tasks, as measured by matrix rank, can be related to the combinatorial complexity of the underlying hyperplane arrangements, potentially revealing a deep connection between algebraic topology and communication complexity.

Taxonomy category: INNOVATIVE_ALGEBRAIC_TOPOLOGY_IN_COMMUNICATION_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 11417e5aeaacab16

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each communication complexity instance with rank R , if the minimal local induction dimension (LID) of its associated hyperplane arrangement over the Boolean ring is $LID/R^2 > 1$ for any seed or $LID/R^2 \leq 1$ for all tested seeds with $n \leq 40$, then the conjecture is supported or falsified, respectively.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Local Induction Dimension AND Boolean Ring AND Communication Complexity Rank - Hyperplane Arrangement IN Boolean Algebra AND Upper Bound ON Matrix Rank - Communication Complexity Rank LIMITS ON Local Induction Dimension Boolean Algebra

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            if pivot == 0:
                continue
            for j in range(n):
                A[i][j] /= pivot
            for j in range(m):
                if j != i:
                    factor = A[j][i]
                    for k in range(n):
                        A[j][k] -= factor * A[i][k]
        return A

    def matrix_rank(A):
        m, n = len(A), len(A[0])
        rank = 0
        for i in range(m):
```

```

        if any(A[i]):
            rank += 1
    return rank

def minimal_local_induction_dimension(A):
    m, n = len(A), len(A[0])
    A_augmented = [row + [1] for row in A]
    A_rref = gaussian_elimination(A_augmented)
    free_vars = n - sum(1 for row in A_rref if any(row))
    return free_vars

def generate_communication_matrix(rank):
    matrix = []
    while len(matrix) < rank:
        row = [random.choice([0, 1]) for _ in range(rank)]
        if row not in matrix:
            matrix.append(row)
    return matrix

n_max = 40
instances_tested = 0
total_metric_value = 0.0
conjecture_holds = True
counterexample = ""

for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5):
        R = random.randint(1, min(n, 10))
        comm_matrix = generate_communication_matrix(R)
        lid = minimal_local_induction_dimension(comm_matrix)
        instances_tested += 1
        total_metric_value += lid / (R ** 2)

mean_metric_value = total_metric_value / instances_tested
if any(lid / (R ** 2) > 1 for R, lid in zip([5, 10, 15, 20, 30, 40], [minimal_local_induction_
    conjecture_holds = False
    counterexample = "LID/R^2 > 1 for some rank"

return {
    "metric_name": "LID/R^2",
    "metric_value": mean_metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and any(r["counterexample"] == "LID/R^2 > 1" for r in results):
    print(f"RESULT: FALSIFIED counterexample=\"LID/R^2 > 1 for some rank\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

{'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.0014467592592592592, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.002259259259259259, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.00041152263374485596, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.001534438775510204, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.0005208333333333333, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.0008541666666666667, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.0028677721088435373, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.0, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
TRIAL: {'metric_name': 'LID/R^2', 'metric_value': 0.002259259259259259, 'instances_tested': 30, 'n_max': 40, 'conjecture_holds': True, 'counterexample': ''}
RESULT: SUPPORTED mean=0.0012426627544021723 std=0.0011929847695617858 support_fraction=1.0

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test only considers a small range of n (up to 40) and uses a fixed number of trials per value of n , which may not be sufficient to capture the behavior of the conjecture over a larger scale. The metric 'LID/R²' could be trivially bounded by construction if the communication matrix has a rank that is always less than or equal to $n/2$.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge | original judge: The test results indicate that for all tested instances with $n \leq 40$, the ratio LID/R² is less than or equal to 1, which meets the support condition o | next: Further investigation should be conducted to verify the conjecture over a larger range of n and with varying communication matrix ranks to ensure its general validity.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13935
2	preregistration	ollama_remote	glm4:latest	0	0	9523
3	novelty	ollama_remote	glm4:latest	0	0	8917
4	novelty	ollama_remote	glm4:latest	0	0	8869
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13250
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11286
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12156
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11902
9	critic	ollama_remote	glm4:latest	0	0	14335
10	judge	ollama_remote	glm4:latest	0	0	9424

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 113598 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5b9ce10bce33.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5b9ce10bce33.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5b9ce10bce33.tar.gz` (if generated)